

AI Skills Track

Product Requirements Document
Web Platform — Full Specification · v1.0

Product	AI Skills Track
Document type	Product Requirements Document (PRD)
Version	1.0
Platform	Web application (browser-based)
Status	Final — ready for development
Audience	Development team

Table of Contents

Table of Contents.....	2
1. Product Overview.....	4
1.1 What Is AI Skills Track.....	4
1.2 Problem Statement.....	4
1.3 Product Vision	4
1.4 Success Metrics	4
2. Users & Personas	6
2.1 Primary Users — Students	6
2.2 Secondary Users	6
3. Scope.....	7
3.1 In Scope — Phase 1 (Pilot).....	7
3.2 Out of Scope — Phase 1	7
4. Curriculum Structure	8
4.1 Four-Year Track Overview.....	8
4.2 Year 1 — AI Foundations (10 Modules).....	8
4.3 Year 2 — AI Builder (10 Modules)	9
4.4 Year 3 — AI Professional (10 Modules).....	11
4.5 Year 4 — AI Career Accelerator (10 Modules).....	12
5. The Certification Loop	15
5.1 Overview	15
5.2 GitHub Verification — Technical Specification	15
Required verification steps.....	15
GitHub API endpoints required.....	16
5.3 Certificate Specification	16
6. Feature Requirements	18
6.1 Authentication & Onboarding	18
Registration.....	18
Login.....	18
GitHub OAuth connection	18
6.2 Student Dashboard.....	18
6.3 Lesson Content Delivery.....	18
6.4 Project Submission.....	19
Project brief page.....	19
Submission form	19
6.5 Quiz Engine	19

Structure	19
Scoring and pass/fail	19
6.6 Certificate Generation	20
6.7 Admin Dashboard	20
Overview panel	20
Student management	20
Content management	20
Placement cell view (read-only role)	21
7. Screen List & User Flows	22
7.1 Complete Screen List	22
7.2 Core User Flows	22
Flow 1 — New student onboarding	22
Flow 2 — Module completion (the certification loop)	23
Flow 3 — Admin reviews a student	23
Flow 4 — Placement cell views Year 4 completers	23
8. Technical Requirements	24
8.1 Recommended Tech Stack	24
8.2 API Endpoints — Minimum Required	24
8.3 Security Requirements	25
8.4 Performance Requirements	25
9. Design System	26
9.1 Visual Identity	26
9.2 Key UI Components	26
Student dashboard — module status dots	26
Certificate PDF design	27
10. Non-Functional Requirements	28
11. Development Milestones	29
12. Open Questions	30

1. Product Overview

1.1 What Is AI Skills Track

AI Skills Track is a structured, 4-year, campus-native AI certification platform designed for undergraduate students. The platform delivers year-appropriate AI learning content, enables students to build and submit real projects through GitHub, assesses understanding through structured quizzes, and issues verifiable digital certificates upon completion of each module.

The core differentiator is the certification loop: no student can receive a certificate by watching content alone. Every certificate requires a verified GitHub project submission, proving that real, employer-checkable work was completed — not passive consumption.

1.2 Problem Statement

Undergraduate students graduate without practical, demonstrable AI skills. Existing options fail them in three specific ways:

- Academic AI electives are theory-heavy and disconnected from employer expectations.
- Generic online platforms give the same content to first-year and final-year students with no structure tied to academic stage.
- Professional AI certification programmes are priced at ₹20,000–₹4,00,000+ — designed for working professionals, not students.

The result: students leave with certificates but no employer-verifiable project work behind them.

1.3 Product Vision

A single web platform where a student can enrol in their year-appropriate AI track, work through structured lessons, build and push a real project to GitHub, pass an auto-graded assessment, and receive a verifiable certificate — all within one session, without switching tools or platforms.

1.4 Success Metrics

Metric	Pilot target	Full launch target
Course completion rate per year	≥ 60%	≥ 75%
GitHub projects submitted per enrolled student	≥ 1 per module	≥ 2 per module
Quiz pass rate (first attempt)	≥ 50%	≥ 65%
Student satisfaction score (post-course survey)	≥ 4.0 / 5.0	≥ 4.3 / 5.0
Certificates issued per enrolled student	≥ 55%	≥ 70%
Time from enrolment to first project submission	< 7 days	< 5 days

2. Users & Personas

2.1 Primary Users — Students

Persona	Year	Core need	Willingness to pay
Explorer	Year 1 — any branch	AI literacy, no intimidation, first real project	Low — needs low-stakes entry point
Builder	Year 2	Real projects, GitHub portfolio, deployable apps	Moderate — values tangible output
Job Seeker	Year 3	Industry-relevant AI skills before placement season	High — urgency drives willingness to pay
Candidate	Year 4	Interview prep, polished portfolio, placement readiness	Highest — immediate, high-stakes outcome

2.2 Secondary Users

- College Administrator / Principal — views aggregate enrolment and completion data for institutional reporting.
- Placement Cell — reads Year 3 and Year 4 student portfolios and GitHub profiles to share with recruiting companies.
- Platform Admin — manages content uploads, student records, certificate issuance, and technical operations.

3. Scope

3.1 In Scope — Phase 1 (Pilot)

The pilot platform must cover all four year-tracks end-to-end. It is not a clickable mockup — a real student must be able to complete the full certification loop (lesson → project → quiz → certificate) for at least one module per year-track.

- Student registration, login, and GitHub account connection.
- Student dashboard showing enrolled track, progress, and next module.
- Lesson content delivery for at least one module per year-track (4 modules minimum for pilot).
- GitHub project submission flow with API-based verification.
- Auto-graded quiz engine (MCQ + short theory questions).
- PDF certificate generation with unique verification code.
- Public certificate verification URL.
- Admin dashboard: enrolment data, completion rates, project submission status.
- Placement cell read-only portal (Year 3 and Year 4 completers).
- Responsive design: desktop (1280px+) and tablet (768px+).

3.2 Out of Scope — Phase 1

- Mobile native app (iOS/Android) — Phase 2.
- Payment gateway integration — pilot is free; payment collected offline if applicable.
- Peer review or mentor review queues — Phase 2.
- Live video or synchronous sessions — not part of the platform.
- Multi-institution support — Phase 2.
- Gamification (XP, badges, Boss Battles) — Phase 2.
- Industry certification alignment badges (NVIDIA DLI, AWS, Microsoft) — Phase 2.

4. Curriculum Structure

4.1 Four-Year Track Overview

Year	Track name	Target student	Core outcome
1	AI Foundations	Any branch — no coding background required	AI literacy, prompting, vibe coding, responsible use, first real project
2	AI Builder	Continuation of Year 1 — ready to code	Python for AI, APIs, ML basics, 2–3 GitHub-verified projects, deployed app
3	AI Professional	Year 2 completers — going deeper technically	NLP, LLMs, computer vision, AI agents, full-stack AI application
4	AI Career Accelerator	Final-year students — production-level AI	RAG, fine-tuning, MLOps, AI system design, production capstone

4.2 Year 1 — AI Foundations (10 Modules)

Module	Syllabus (what they learn)	Project (GitHub submission)	Quiz focus
1.1	History of AI, types of AI (generative, predictive, recommendation, computer vision, NLP), real examples students already use, capabilities vs limitations.	GitHub repo with a structured document: 5 real AI prompts with documented outputs and analysis.	AI types, capabilities, limitations. Theory: Explain in your own words what a language model does.
1.2	Anatomy of a prompt (role, context, task, format), zero-shot vs few-shot prompting, chain-of-thought, iteration, spotting hallucinations.	Prompt library: GitHub repo with 10 optimised prompts for a chosen domain, before/after comparisons.	Prompt components, failure modes. Theory: Why does prompt specificity matter? Give an example.
1.3	Advanced prompting — meta-prompting, self-consistency, structured outputs. Building a personal prompt library across study, work, and creative use cases.	Advanced prompt portfolio: 15 prompts across 3 domains with documented iteration history.	Advanced prompt techniques. Theory: What is self-consistency prompting and when would you use it?
1.4	AI tools landscape — ChatGPT, Gemini, Claude, Copilot, Canva AI, Grammarly, Perplexity. How to evaluate tools. Responsible use, privacy, academic integrity, bias.	AI toolkit report pushed to GitHub as markdown: 3 tools reviewed, use cases tested, pros/cons, ethical considerations.	Tool categories, responsible use scenarios, bias examples. Theory: Describe a situation where AI use would be inappropriate.
1.5	What is vibe coding — building software with plain	Introduction to vibe coding tools — student explores	Vibe coding concept, tools, and limitations.

Module	Syllabus (what they learn)	Project (GitHub submission)	Quiz focus
	English. Tools: Bolt.new, v0.dev, Cursor, Replit AI. The shift from 'I can't code' to 'I can build'.	Bolt.new and documents 3 things they tried to build.	Theory: What is the difference between prompting an AI and vibe coding?
1.6	Build with vibe coding — live walkthrough. Student builds a working personal portfolio website or simple web tool using only plain English descriptions. Deploy it.	Deployed personal portfolio website or simple web tool built with Bolt.new or v0.dev. GitHub repo + live URL.	Vibe coding workflow, debugging with AI. Theory: Describe one limitation of vibe coding you encountered or can anticipate.
1.7	AI tools for productivity — Notion AI, scheduling, email, task management. Introduction to no-code automation: Zapier, Make.com basics. Identifying repetitive tasks.	Automation project: identify one repetitive task, build a simple automation using Zapier or Make.com free tier, document it.	Automation concepts, tool connections. Theory: What type of tasks are most suitable for AI automation?
1.8	AI across industries — how AI is changing engineering, business, healthcare, law, arts, and education right now. Real Indian company examples. Jobs being created vs changing.	Industry research report: how AI is changing one specific industry the student cares about. 3 real company examples.	Industry-specific AI applications. Theory: How will your degree field be different in 10 years because of AI?
1.9	Critical thinking about AI — what AI gets wrong, bias and misinformation, deepfakes, over-reliance, privacy, academic integrity in professional settings.	Critical analysis document: evaluate one AI-generated piece of content for accuracy, bias, and reliability.	AI limitations, bias, misinformation. Theory: You receive an AI-generated research summary. How would you verify it?
1.10	Future-proofing yourself — AI-native careers, salary data for AI roles in India, reading a JD for AI skill requirements, personal 6-month AI action plan.	Personal AI roadmap: written 6-month plan specific to the student's degree field and career goals. GitHub doc.	Career trends, AI role types, JD analysis. Theory: List 3 specific AI skills that would make you more employable in your field.

4.3 Year 2 — AI Builder (10 Modules)

Module	Syllabus (what they learn)	Project (GitHub submission)	Quiz focus
2.1	Why code when you can vibe code? What Python adds that plain English prompting cannot. Variables, data types, lists, dictionaries, loops, functions. Google Colab setup.	First Python script: a simple data processor that reads a list, filters it, and outputs results. Google Colab notebook on GitHub.	Python syntax, data types, Google Colab basics. Theory: What is a pre-trained model and why is it useful for beginners?

Module	Syllabus (what they learn)	Project (GitHub submission)	Quiz focus
2.2	NumPy, Pandas, Matplotlib — the three essentials. Reading CSVs, cleaning data (nulls, duplicates, wrong types), grouping, aggregating, plotting charts.	Data cleaning project: take a messy public dataset, clean it, produce 3 labelled charts with written commentary. Full notebook on GitHub.	Pandas functions, chart types, data cleaning. Theory: A dataset has 30% null values. What are your options?
2.3	What an API is, HTTP requests (GET/POST), JSON, API keys, reading documentation. Authentication best practices — never commit an API key to a public repo.	API exploration project: call a free public API (weather, jokes, news), process the JSON response, and display it. GitHub repo with .env example.	API concepts, HTTP methods, JSON structure. Theory: What is an API key and why must it never be in a public repo?
2.4	Gemini or OpenAI free-tier API — building a Python chatbot that takes user input in a loop and returns AI responses. Conversation history handling.	Chatbot project: Python chatbot using a free AI API, accepts user input, returns AI response, handles 5+ turns. GitHub repo with README.	API parameters, conversation structure. Theory: What does 'temperature' control in an LLM API call?
2.5	What machine learning is — training, testing, features, labels, models. Scikit-learn basics. Linear regression. Classification (logistic regression, decision tree).	ML classifier: choose a public dataset, train a classifier, evaluate it (accuracy, precision, recall). Notebook on GitHub with findings documented.	ML concepts, model types, evaluation metrics. Theory: What is the difference between training data and test data?
2.6	Neural networks — neurons, layers, weights, activation functions, forward pass, backpropagation explained simply. Introduction to TensorFlow/Keras. Training a simple model.	Neural network project: train a simple neural network on MNIST or a similar dataset, document the accuracy and what changes improved it.	Neural network components, training concepts. Theory: What is overfitting and how would you detect it?
2.7	Pre-trained models — what they are, why they exist. HuggingFace pipelines for NLP tasks: text classification, sentiment analysis, summarisation, translation.	NLP project: use a HuggingFace pipeline to classify or summarise a dataset of your choice. Full pipeline in a GitHub repo with results documented.	Pre-trained model concepts, HuggingFace pipeline usage. Theory: When would you use a pre-trained model vs training from scratch?
2.8	Data visualisation for communication — choosing the right chart, labelling, annotations, making data tell a story. Matplotlib, Seaborn, Plotly basics.	Visualisation project: take a public dataset, produce 5 charts that tell a coherent story. GitHub repo with markdown explanation of each chart.	Chart types, visual communication. Theory: You need to show how a metric changed over time. What chart type and why?
2.9	End-to-end AI project thinking — problem definition, data sourcing, model selection, building,	Mini AI project: define a problem, build an AI solution using tools from modules 2.1–	Project structure, documentation. Theory: Describe your project's

Module	Syllabus (what they learn)	Project (GitHub submission)	Quiz focus
	testing, documentation. Streamlit for simple UIs.	2.8, wrap it in a Streamlit app. GitHub repo + live link.	problem statement and approach.
2.10	Deployment — making a Python AI project accessible to others. Streamlit Cloud, Hugging Face Spaces (free hosting). Writing a strong README. GitHub portfolio best practices.	Deployed project: take the Module 2.9 project and deploy it with a public URL. Update GitHub README with setup instructions, screenshots, and live link.	Deployment concepts, README structure. Theory: Why does a live deployment link matter more than just code in a GitHub repo?

4.4 Year 3 — AI Professional (10 Modules)

Module	Syllabus (what they learn)	Project (GitHub submission)	Quiz focus
3.1	Advanced Python — OOP (classes, objects, inheritance), error handling, file I/O, virtual environments, clean code principles, writing documented Python.	Refactored project: take any previous Year 2 project and refactor it using OOP principles. Document the changes and why they improve the code.	OOP concepts, error handling, clean code. Theory: What is the difference between a class and an instance?
3.2	Deep data analysis — EDA as a structured process, feature engineering, statistical basics (mean, median, distribution, correlation), handling large files, merging datasets.	EDA project: structured exploratory analysis of a complex public dataset. Feature engineering, correlation matrix, 5+ charts, written findings.	EDA process, statistical concepts. Theory: What is feature engineering and why does it matter for model performance?
3.3	ML going deeper — random forests, gradient boosting (XGBoost), clustering (K-means), dimensionality reduction (PCA), model evaluation (confusion matrix, F1, ROC-AUC).	ML comparison project: train 3 different models on the same dataset, compare evaluation metrics, explain which model you'd use and why.	Advanced ML algorithms, evaluation metrics. Theory: What is the difference between precision and recall?
3.4	Deep learning going deeper — CNNs for image classification, transfer learning (using a pre-trained CNN for a new task), RNNs conceptually, GPU usage basics.	Transfer learning project: use a pre-trained CNN (ResNet or MobileNet) to classify a custom image dataset. GitHub repo with training logs.	CNN architecture, transfer learning. Theory: Why is transfer learning more practical than training a CNN from scratch?
3.5	NLP deep dive — tokenisation, TF-IDF, sentiment analysis, named entity recognition, transformers architecture (attention mechanism explained intuitively),	NLP pipeline project: build a complete NLP pipeline (cleaning, tokenising, classifying, summarising) on a real text dataset. GitHub repo.	NLP pipeline, transformer concepts. Theory: What is the attention mechanism and why did it change NLP?

Module	Syllabus (what they learn)	Project (GitHub submission)	Quiz focus
	HuggingFace advanced usage.		
3.6	LLMs professionally — system prompts, temperature/top-p parameters, prompt chaining, structured outputs with Pydantic, function calling basics, LLM caching, cost optimisation.	LLM pipeline project: build a multi-step LLM pipeline using prompt chaining and structured JSON outputs for a defined business problem.	LLM parameters, structured outputs. Theory: What is prompt chaining and when would you use it over a single prompt?
3.7	Computer vision — images as data (pixels, channels), OpenCV for processing, YOLO for object detection, CLIP for image-text matching, real use cases.	Computer vision project: build an object detection or image classification pipeline using a pre-trained model. GitHub repo with demo images and results.	Computer vision concepts, model types. Theory: What is object detection and how is it different from image classification?
3.8	AI agents — what makes an agent different from a chatbot (planning, tool use, memory, self-correction). Langchain/LangGraph basics. Giving agents tools (web search, file reading, API calls).	AI agent project: build a simple agent that can use at least 2 tools (e.g. web search + summarisation) to complete a research task autonomously.	Agent architecture, tool use. Theory: What is the ReAct pattern in AI agents?
3.9	Full-stack AI applications — FastAPI for AI backends, connecting a React or HTML frontend to an AI backend, user authentication basics, SQLite for data persistence.	Full-stack AI app: a web application with a real UI, an AI feature, user input handling, and data stored in a database. GitHub repo + live URL.	FastAPI concepts, frontend-backend connection. Theory: Why is a REST API a better interface for an AI model than direct function calls from a frontend?
3.10	Year 3 capstone — end-to-end AI project combining technical depth with a real use case. Problem scoping, solution design, build, deploy, document, present.	Advanced capstone: a full-stack AI application with clear problem statement, AI component, deployed live URL, 5-minute screen-recorded demo video, and comprehensive README.	End-to-end project thinking, documentation. Theory: Describe your capstone's architecture — what components interact and how?

4.5 Year 4 — AI Career Accelerator (10 Modules)

Module	Syllabus (what they learn)	Project (GitHub submission)	Quiz focus
4.1	RAG deep dive — the full pipeline: document ingestion, chunking strategies, embedding models, vector databases	RAG system: a chatbot that answers questions about a real document (uploaded PDF). Full pipeline on GitHub	RAG pipeline components, vector databases, chunking. Theory: What is the difference between

Module	Syllabus (what they learn)	Project (GitHub submission)	Quiz focus
	(Pinecone, Chroma, FAISS), retrieval, reranking, generation with context injection. Evaluating RAG quality.	with README explaining each component.	similarity search and MMR in RAG retrieval?
4.2	Advanced LLM engineering — transformer internals (attention, embeddings, context window), advanced prompting (tree of thought, meta-prompting), multi-modal LLMs, LLM routing, comparing frontier models.	LLM evaluation project: compare 3 frontier models (GPT-4o, Gemini, Claude) on the same task. Structured evaluation with scoring criteria. GitHub report.	Transformer concepts, advanced prompting. Theory: What is the context window and why does it matter for RAG design?
4.3	AI agents and multi-agent systems — ReAct and Plan-and-Execute architectures, LangGraph for stateful agents, CrewAI for multi-agent collaboration, memory systems (short-term, long-term, entity).	Multi-agent system: build a 2-agent pipeline (e.g. researcher agent + writer agent) that collaborates to produce a structured output from a web query.	Agent architectures, memory systems. Theory: What is the difference between a single agent with tools and a multi-agent system?
4.4	Fine-tuning — when to fine-tune vs use RAG vs prompt engineer (critical decision framework). LoRA and QLoRA explained. Dataset preparation. Fine-tuning a small open-source model on Google Colab or Kaggle.	Fine-tuning project: fine-tune a small model (Llama or Mistral) on a custom dataset. Compare fine-tuned vs base model outputs. GitHub repo with training logs.	Fine-tuning concepts, LoRA. Theory: When would you choose fine-tuning over RAG for a production use case?
4.5	Vector databases deep dive — embedding models (Sentence Transformers, BGE, OpenAI), HNSW indexing, approximate nearest neighbour search, hybrid search (vector + keyword), metadata filtering.	Semantic search engine: build a search engine over a dataset of your choice using a vector database and embedding model. GitHub repo + deployed demo.	Vector database internals, embedding models. Theory: What is hybrid search and when is it better than pure vector search?
4.6	MLOps — experiment tracking (MLflow or W&B), model versioning, CI/CD for ML (GitHub Actions), monitoring (data drift, model drift), Docker basics, deployment on cloud (AWS or GCP basics).	MLOps pipeline: set up experiment tracking for a previous ML project, add a GitHub Actions CI check, write a Dockerfile. GitHub repo with full setup.	MLOps concepts, experiment tracking. Theory: What is model drift and how would you detect it in production?
4.7	AI system design — latency, throughput, scalability, cost. Designing for 100,000 users.	System design document: write a full system design for a production AI application (e.g.	System design principles, caching. Theory: Design a RAG system for 100,000

Module	Syllabus (what they learn)	Project (GitHub submission)	Quiz focus
	Caching strategies (semantic, response, embedding). Fallback strategies. Async processing for AI workloads.	customer support bot at scale). GitHub markdown doc with architecture diagram.	concurrent users. What changes compared to a prototype?
4.8	AI security and safety — prompt injection attacks, jailbreaking, data poisoning, PII leakage, content moderation, AI red-teaming, guardrails (Guardrails AI or NeMo), compliance (DPDP Act, GDPR for AI).	Security audit: perform a structured red-team evaluation of one of your own AI projects. Document vulnerabilities found and mitigations applied.	AI security concepts, attack types. Theory: What is a prompt injection attack and how would you defend against it?
4.9	Portfolio polish and placement readiness — GitHub profile best practices, writing strong READMEs, adding demos, clean commit history, personal portfolio website, LinkedIn for AI roles, resume AI skills section.	Portfolio audit: refactor 2 previous projects (updated READMEs, deployment links, demo recordings). Update GitHub profile README. GitHub + LinkedIn profile link submitted.	GitHub best practices, portfolio presentation. Theory: A recruiter has 60 seconds on your GitHub. What 3 things must they see?
4.10	Year 4 production capstone — the defining project. Must demonstrate at least 3 of: RAG, agents, fine-tuning, MLOps, full-stack deployment, multi-modal AI, real users or real data.	Production capstone: full GitHub repo, live deployed application, technical write-up, 7-minute demo video. Reviewed by senior reviewer. Top projects featured on platform.	End-to-end architecture, production thinking. Theory: Explain your capstone's architecture, the tradeoffs you made, and one thing you would do differently.

5. The Certification Loop

5.1 Overview

Every module across all four year-tracks follows the same four-step certification loop. This loop is the core product mechanic and must function identically across all tracks. No student can skip a step. The platform enforces the sequence.

Step	Name	What happens	How the next step unlocks
1	Syllabus & Training	Student works through all lesson sections for the module — text content, diagrams, and embedded video. Content covers all concepts needed to complete the project.	Student clicks 'Mark as complete' at the bottom of all lesson sections. Platform records completion and unlocks the project submission step.
2	Project Submission	Student builds a real project using what they learned, pushes the completed work to their GitHub repository, and submits the repo URL (or document/video link where applicable) on the platform.	Platform calls the GitHub REST API to verify: (a) repo exists and is public, (b) at least one commit beyond the template baseline exists, (c) commit is authored by the authenticated student's GitHub account. Verified = quiz unlocked.
3	Quiz	Student completes a 10-question module quiz. 70% of questions are MCQ (single correct answer). 30% are theory (short written explanation, max 200 words). Pass mark: 60% overall. MCQ is auto-graded immediately. Theory questions are reviewed by admin within 48 hours.	Student who passes MCQ at 60%+ is provisionally passed. Certificate issued automatically. Admin reviews theory within 48 hours and can flag if quality is unacceptable.
4	Certificate	Platform generates and issues a signed PDF certificate containing: student full name, track name and year, module name, date of completion, unique alphanumeric verification code, and a public verification URL. Certificate is stored permanently and downloadable at any time.	Permanent. Certificate accessible via public URL for minimum 5 years. Any employer can verify at the verification URL without logging in.

5.2 GitHub Verification — Technical Specification

GitHub verification is the most technically critical component of the platform. It must be implemented with the GitHub REST API — not simulated or manually reviewed in Phase 1.

Required verification steps

1. Student connects their GitHub account via OAuth (GitHub OAuth App) during onboarding.

2. Platform provides a repository template (pre-created on the platform's GitHub organisation) that the student forks for each project module.
3. Student completes the project, pushes commits to their forked repo, and submits the repo URL on the platform.
4. Platform calls the GitHub REST API to verify all three conditions: repo exists and is public, has at least one commit beyond the template baseline, and the commit was authored by the authenticated student's GitHub username.
5. If verification passes: student is marked 'Project submitted' and the quiz is unlocked.
6. If verification fails: platform shows a specific error message identifying which condition failed (repo not found / no new commits / wrong account). Student can resubmit after fixing the issue.

GitHub API endpoints required

- GET /repos/{owner}/{repo} — confirms the repository exists and is publicly accessible.
- GET /repos/{owner}/{repo}/commits — confirms at least one commit exists beyond the template's initial commit baseline.
- GET /repos/{owner}/{repo}/commits?author={student_github_username} — confirms the student's account authored the commit.

Non-negotiable

In the pilot, admin manual override is acceptable if the GitHub API call fails due to a technical error — but the API verification must be attempted first. Manual override is the fallback, not the default. GitHub verification is the core trust mechanism of the entire product.

5.3 Certificate Specification

Certificates are generated as PDF documents (not images) and must include:

- Student full name as entered during registration.
- Track name and year (e.g. 'AI Foundations — Year 1').
- Module name completed.
- Date of completion.
- Unique alphanumeric verification code (e.g. AIST-2026-00342).
- Public verification URL (e.g. verify.aiskillstrack.in/AIST-2026-00342) that returns certificate details when visited — no login required.
- Platform logo.

Year 4 certificates use a distinct, premium visual design (gold accent instead of green) to reflect the advanced nature of the track. They must be visually distinguishable from Years 1–3 certificates.

6. Feature Requirements

6.1 Authentication & Onboarding

Registration

- Required fields: full name, email address, institution roll number, year of study (1–4), branch/department, phone number, password.
- Year of study auto-assigns the student to the correct year-track.
- Email verification required via OTP before platform access is granted.
- Roll number uniqueness enforced at database level.

Login

- Email + password login.
- Forgot password flow via email OTP.
- Session persistence — student remains logged in for 7 days unless they log out explicitly.

GitHub OAuth connection

- Prompted after first login — students must connect their GitHub account before accessing any project module.
- If no GitHub account exists, platform shows a step-by-step guide to create one.
- GitHub username stored against the student record and used for all subsequent verification API calls.

6.2 Student Dashboard

The student dashboard is the home screen after login. It must display:

- Student name and current year/track.
- Overall progress bar: modules completed vs total modules in track.
- Current module card — the next module to complete, with a 'Continue' CTA.
- All-modules list: each module showing status (Locked / In Progress / Completed), project status (Submitted / Verified / Pending), quiz score (if taken), and certificate download link (if issued).
- GitHub connection status badge — green if connected, amber if not yet connected.
- All four year-track cards — completed tracks show the certificate date; locked tracks show the prerequisite.

6.3 Lesson Content Delivery

- Each module has a dedicated lesson page with structured content sections.

- Content types supported: text with headings and formatted paragraphs, embedded YouTube video (iframe), downloadable PDF resources, syntax-highlighted code blocks, external links.
- Content is authored externally and uploaded via admin CMS — the platform delivers it, not generates it.
- 'Mark as complete' button at the bottom of all lesson sections unlocks the project submission step.
- Lesson progress is saved — if a student leaves and returns, they resume where they left off.

6.4 Project Submission

Project brief page

- Displays: project title, objective, step-by-step instructions, acceptance criteria, GitHub repo template link (forkable), and optional deadline (if set by admin).
- 'I have completed the project' button opens the submission form.

Submission form

- Code projects (Years 2–4): GitHub repo URL field + optional notes field.
- Document projects (Year 1): GitHub repo URL containing the uploaded document.
- Video submissions (Modules 3.10, 4.10): YouTube/Drive video link field.
- Document uploads (Module 4.9): PDF upload (max 5MB) + LinkedIn profile URL field.
- On submission: GitHub verification runs immediately. Result shown within 5 seconds. Specific error message displayed on failure.
- Verified submissions cannot be deleted by the student.

6.5 Quiz Engine

Structure

- Minimum 10 questions per module.
- Question types: MCQ — single correct answer (70%), theory — short written response max 200 words (30%).
- MCQ questions displayed one at a time with a progress indicator.
- Theory questions displayed after all MCQ questions, as a text entry field.
- No timer in Phase 1.

Scoring and pass/fail

- Pass mark: 60% overall. MCQ auto-graded immediately. Theory reviewed by admin within 48 hours.
- Student who passes MCQ at 60%+ is provisionally passed — certificate issued. Theory reviewed within 48 hours.
- If MCQ failed: score shown, correct answers revealed, 'Retake quiz' button available (unlimited retakes in Phase 1).
- All attempt scores and theory answers stored against the student record.

6.6 Certificate Generation

- Certificate PDF generated automatically when the student passes the quiz.
- Two PDF templates: standard (Years 1–3, green accent) and premium (Year 4, gold accent).
- Certificate PDF stored on server, accessible via permanent URL.
- Student can download from their dashboard at any time after issuance.
- Each certificate has a unique verification code queryable at a public URL — no login required to verify.
- Admin can revoke a certificate (mark as invalid) if academic fraud is confirmed.

6.7 Admin Dashboard

Overview panel

- Total enrolled students by year/track.
- Total modules completed across all students.
- Total certificates issued.
- Completion rate per track.

Student management

- Full student list: name, roll number, year, track, modules completed, GitHub username, last active date.
- Search and filter by year, track, completion status, GitHub connection status.
- Click into individual student: full progress view, all quiz scores and theory answers, all project GitHub links, all certificates.
- Manual override: admin can mark a project as verified if GitHub API fails for a legitimate reason. Override logged with timestamp and admin user ID.

Content management

- Upload and edit lesson content per module (rich text editor + video embed + PDF upload).

- Upload and edit quiz questions per module (add, edit, delete MCQ and theory questions).
- Upload and edit project briefs per module.
- Set and edit module deadlines.

Placement cell view (read-only role)

- Separate login for placement cell staff — read-only, no edit access.
- View Year 3 and Year 4 completers only.
- See each student's name, GitHub profile link, capstone project link, demo video link, and certificate verification URL.
- Cannot edit or delete any student data.

7. Screen List & User Flows

7.1 Complete Screen List

Screen	Route (suggested)	Description
Landing / home	/	Public marketing page — product pitch, curriculum overview, enrol CTA
Registration	/register	New student sign-up form
Email verification	/verify-email	OTP entry screen
Login	/login	Email + password login form
GitHub connect	/connect-github	OAuth flow — prompted after first login
Student dashboard	/dashboard	Home screen: track, progress, next module
Module — lesson	/module/:id/lesson	Lesson content for the selected module
Module — project	/module/:id/project	Project brief and GitHub submission form
Module — quiz	/module/:id/quiz	Quiz engine — question by question
Quiz result	/module/:id/result	Pass/fail screen with score and next step
Certificate view	/certificate/:code	Public certificate display — no login required
Certificates list	/dashboard/certificates	Student's list of all earned certificates with download links
Profile	/dashboard/profile	Student profile, GitHub connection status, progress summary
Admin — overview	/admin	Admin dashboard overview panel
Admin — students	/admin/students	Student list, search, and filter
Admin — student detail	/admin/students/:id	Individual student progress, submissions, quiz history
Admin — content	/admin/content	CMS for lessons, project briefs, and quiz questions
Admin — certificates	/admin/certificates	All issued certificates, revoke option
Placement cell view	/placement	Read-only Year 3/4 completer view for placement staff
Forgot password	/forgot-password	Password reset via email OTP
404 / error	/404	Error and not-found page

7.2 Core User Flows

Flow 1 — New student onboarding

1. Student visits the landing page and clicks 'Enrol now'.
2. Completes the registration form (name, email, roll number, year, department, password).
3. Receives email OTP, enters it on /verify-email.
4. Redirected to /connect-github — connects GitHub account via OAuth.
5. Redirected to /dashboard — sees their year-track and first module unlocked.

Flow 2 — Module completion (the certification loop)

1. Student clicks 'Start' or 'Continue' on their current module from /dashboard.
2. Reads lesson content on /module/:id/lesson. Clicks 'Mark as complete'.
3. Proceeds to /module/:id/project — reads project brief, forks the GitHub template.
4. Completes the project, pushes to GitHub, submits the repo URL on the platform.
5. Platform runs GitHub API verification. Success or specific error message shown.
6. 'Take quiz' button unlocked. Student navigates to /module/:id/quiz.
7. Answers all MCQ and theory questions. Submits the quiz.
8. Passes ($\geq 60\%$ MCQ): certificate generated and download link shown on /module/:id/result.
9. Dashboard updates — module marked complete, next module unlocked.

Flow 3 — Admin reviews a student

1. Admin logs into /admin.
2. Views overview panel — completion rates, enrolment numbers.
3. Navigates to /admin/students, searches for a specific student.
4. Views full progress, quiz scores, theory answers, GitHub project links.
5. If GitHub verification failed for a legitimate reason, admin applies manual override with a note.

Flow 4 — Placement cell views Year 4 completers

1. Placement cell staff logs into /placement.
2. Views list of all Year 4 certificate holders.
3. Clicks a student to see GitHub profile, capstone project link, demo video link, and certificate verification URL.
4. Copies relevant links to share with a recruiting company.

8. Technical Requirements

8.1 Recommended Tech Stack

Layer	Recommended	Notes
Frontend	React.js (Next.js)	SSR for SEO on public pages, CSR for the application. Or plain React if preferred.
Backend	Node.js + Express or Next.js API routes	REST API — must be fully documented for future development.
Database	PostgreSQL	Relational structure suits student records, progress, quiz scores, and certificates.
Authentication	JWT tokens + email OTP	GitHub OAuth via GitHub OAuth App. Do not use a third-party auth service that may rate-limit.
File storage	AWS S3 or Cloudflare R2	Certificate PDFs, uploaded documents, resource PDFs.
Certificate PDF generation	Puppeteer or PDFKit	Puppeteer preferred — allows HTML/CSS template design with full visual control.
GitHub integration	GitHub REST API v3	Three endpoints sufficient (see Section 5.2). No GraphQL required.
Email	SendGrid or Resend	OTP emails, certificate issued notifications.
Hosting	Vercel (frontend) + Railway or Render (backend)	Or any equivalent cloud deployment. Must support HTTPS.
In-browser IDE	Monaco Editor (CDN)	Load from cdnjs.cloudflare.com — same editor as VS Code. Required for Year 2+ coding modules.

8.2 API Endpoints — Minimum Required

Endpoint	Method	Description
POST /auth/register	POST	Create new student account
POST /auth/verify-email	POST	Verify email OTP
POST /auth/login	POST	Authenticate student, return JWT
GET /auth/github/connect	GET	Initiate GitHub OAuth flow
GET /auth/github/callback	GET	Handle OAuth callback, store GitHub token
GET /student/dashboard	GET	Return student's full progress data
GET /modules/:id	GET	Return module details (lesson, project brief, quiz)
POST /modules/:id/lesson/complete	POST	Mark lesson as read, unlock project step

Endpoint	Method	Description
POST /modules/:id/project/submit	POST	Submit project URL, trigger GitHub API verification
POST /modules/:id/quiz/submit	POST	Submit quiz answers, return score, issue certificate if passed
GET /certificates/:code	GET	Public — return certificate data for verification (no auth required)
GET /admin/students	GET	Admin — return all students with full progress
PATCH /admin/projects/:id/override	PATCH	Admin — manually mark a project as verified with a note
GET /placement/completers	GET	Placement role — return Year 3/4 completers with portfolio links

8.3 Security Requirements

- All API endpoints require a valid JWT token except: POST /auth/register, POST /auth/login, POST /auth/verify-email, GET /certificates/:code.
- Admin and placement cell endpoints require role-based authorisation — a student JWT must not be accepted on admin or placement routes.
- GitHub OAuth tokens stored encrypted in the database. Never exposed in API responses.
- Passwords hashed with bcrypt, salt rounds ≥ 12 .
- All HTTP traffic must use HTTPS — no exceptions in any environment.
- Roll number uniqueness enforced at database level — duplicate registration prevented.
- Rate limiting on authentication endpoints: maximum 10 attempts per IP per 15 minutes.
- All admin override actions logged with timestamp, admin user ID, and reason.

8.4 Performance Requirements

- Page load time (first contentful paint): < 2 seconds on a standard 4G mobile connection.
- GitHub verification API call: must complete within 5 seconds or return a timeout error with a clear message.
- Quiz submission and result display: < 1 second response time.
- Certificate PDF generation: < 10 seconds from quiz pass to PDF available for download.
- Platform must support at least 500 concurrent users without degradation (pilot batch scope).

9. Design System

9.1 Visual Identity

Element	Specification
Primary colour	#1F4E3D (deep forest green) — headings, primary buttons, accents, active states
Secondary colour	#EAF3E6 (pale mint) — callout boxes, card backgrounds, hover states
Accent colour	#3DBA6E (bright green) — CTAs, verified badges, success states
Gold accent	#C9A84C (amber gold) — Year 4 certificates and premium elements only
Text primary	#1A1A18 (near-black) — all body text
Text secondary	#5F5E5A (warm grey) — labels, metadata, captions
Error colour	#791F1F (deep red) — error states, failed verification, failed quiz
Success colour	#1F4E3D (same as primary) — success states, verified badges
Warning colour	#7A4E0B (amber) — pending states, provisional pass
Background light	#FFFFFF — default page background
Background dark	#0A0A0A — login screen, dark mode base
Surface (cards)	#F8FAF8 — card and panel backgrounds
Border	#D9D6CE — all card and input borders
Primary font	Inter (Google Fonts) — all body text, UI labels, buttons
Heading font	Cambria or Inter Bold — all headings
Border radius	12px for cards, 8px for inputs, 4px for buttons, 999px for pills/badges
Shadow	0 2px 8px rgba(0,0,0,0.08) — applied to cards only, not flat elements

9.2 Key UI Components

Student dashboard — module status dots

Status	Visual	Colour
Locked	Grey circle, lock icon	#D1D0CB fill
In Progress	Mint circle, outline	#EAF3E6 fill, #1F4E3D border
Completed	Green circle, check icon	#1F4E3D fill, white check
GitHub verified	Green badge	#EAF3E6 fill, #1F4E3D text, checkmark prefix
Quiz failed	Red badge	#FCEBEB fill, #791F1F text

Certificate PDF design

- A4 landscape orientation.
- Top colour bar: 6px, full width. Green (#1F4E3D) for Years 1–3. Gold (#C9A84C) for Year 4.
- Header: platform logo left, institution name right.
- Centre text: 'This certifies that [Student Name] has successfully completed [Module Name] as part of the AI Skills Track.'
- Student name in primary green (#1F4E3D) at 26pt — gold (#C9A84C) for Year 4.
- Date of completion, verification code in monospace font, and public verification URL at the bottom.
- Double-line decorative border in the primary colour.

10. Non-Functional Requirements

Requirement	Specification
Browser support	Chrome 110+, Firefox 110+, Safari 16+, Edge 110+. Internet Explorer not supported.
Responsiveness	Fully functional on desktop (1280px+) and tablet (768px–1279px). Mobile layout is Phase 2.
Accessibility	WCAG 2.1 Level AA. All images have descriptive alt text. Form inputs have labels. Colour contrast ratio $\geq 4.5:1$ for all text.
Dark mode	Toggle available in the top navigation. Preference saved in local storage. CSS variables used throughout for seamless switching.
Uptime	95% uptime during active hours (8am–10pm IST). Maintenance window: 2am–5am IST.
Data privacy	Student PII (name, email, roll number, phone, GitHub username) stored encrypted at rest. Not shared with any third party. Compliant with India IT Act 2000.
Certificate permanence	Issued certificates must remain accessible and verifiable for a minimum of 5 years from the date of issue.
Database backup	Daily automated backups. Minimum 30-day backup retention. Recovery time objective: < 4 hours.
Audit logging	All admin actions (manual overrides, content edits, certificate revocations) logged with: timestamp, admin user ID, action type, and reason field.
In-browser IDE	Monaco Editor (same as VS Code) loaded from cdnjs.cloudflare.com. Python syntax highlighting enabled by default for Year 2–4 coding modules. Plain text/markdown editor for Year 1 document submissions.

11. Development Milestones

Critical date

The pilot must be live and accessible to the first student batch by 16 July 2026. Any delay past this date pushes the pilot to the following semester.

Milestone	Description	Target date
M1 — PRD sign-off	Development team agrees this PRD is the build specification. No scope changes after this point.	30 June 2026
M2 — Design complete	All screens designed and shared for review. Design system applied consistently.	5 July 2026
M3 — Auth + dashboard live	Registration, login, GitHub OAuth, and student dashboard functional end-to-end.	8 July 2026
M4 — Certification loop live (1 module)	Full 4-step loop working end-to-end for one module in one year-track.	11 July 2026
M5 — All 4 year-tracks live	All 40 modules accessible, full loop functional across all four year-tracks.	14 July 2026
M6 — Admin + placement portal live	Admin dashboard, content CMS, and placement cell read-only view functional.	14 July 2026
M7 — Content uploaded	All lesson content, project briefs, and quiz questions uploaded into the CMS for the pilot modules.	14 July 2026
M8 — UAT complete	All flows tested. Critical bugs resolved. Platform stable.	15 July 2026
M9 — Pilot launch	Platform live. First student batch invited to enrol.	16 July 2026
M10 — Pilot review	Completion rates, GitHub submissions, quiz scores, and feedback reviewed.	31 August 2026

12. Open Questions

The following questions must be resolved before or during development. They do not block the start of build but will affect specific features.

#	Question	Impact	Priority
1	What is the platform domain? (e.g. aiskillstrack.in)	Certificate verification URLs depend on this. Must be decided before M4.	High
2	Is the pilot free or charged (₹0 or ₹500)? If charged, how is payment collected?	Determines whether a payment gateway is needed in Phase 1.	High
3	Who owns and manages the GitHub organisation for repo templates?	GitHub template repos must exist before M4. Delays here delay module launch.	High
4	Are short-answer theory quiz questions manually reviewed or auto-graded in Phase 1?	If manual, the admin dashboard needs a theory review queue with notification.	Medium
5	What is the student institution roll number format and validation regex?	Registration form validation and roll number uniqueness enforcement.	Medium
6	What is the exact signatory title on certificates?	Certificate PDF template design requires this before M2.	Low

This document defines the complete specification for the AI Skills Track web platform. Any developer or development team should be able to build the platform from this document alone without requiring additional clarification on product scope.